Лабораторна робота № 2

з дисципліни «Системне програмування»

на тему «Інтеграція сенсорної підсистеми ToolSafe»

Виконав:

студент групи ІО-32

Голуб Денис Дмитрович

Перевірив:

Гордієнко Нікіта Юрійович

Київ – 2026

Тема: Інтеграція сенсорної підсистеми ToolSafe: підключення, аналіз інтерфейсів і шарова архітектура.

Мета роботи: Підключити всі три сенсори одночасно, підтвердити їхню роботу незалежними цифровими записами, перевірити portable drivers та прикладні правила тестами, визначити межі симуляційної перевірки.

Постановка завдання

За методичкою ЛР2 необхідно обґрунтувати pinout і бюджет живлення, зібрати спільний стенд, виконати ідентифікацію й самоперевірку, записати I2C/UART/GPIO та маркер ADC. Потрібно декодувати ID і burst, вручну розібрати адресні біти, створити NACK, виміряти час, відокремити vendor/board/device/application layers, додати fake tests та власний captured vector, виміряти coverage й налаштувати CI. Результати мають містити component diagram, журнал проблем і межі достовірності.

Хід роботи

Призначення та межі прототипу

Проєкт **ToolSafe** реалізує спостереження за сейфом з інструментами. Використано форму B: модель NUCLEO-C031C6, MPU6050, PIR і аналоговий LDR в одному стенді Wokwi. Середовище: Ubuntu 24.04, Docker, STM32 HAL, Wokwi. Еталонний перелік функцій визначено в PRD першої лабораторної; друга робота додає одночасний збір даних, правила станів і незалежний аналіз сигналів.

Цифрові траси отримано 15.10.2026. Записи, UART і screenshots походять із виконання цієї прошивки. Симулятор не вимірює струм, живлення, електромагнітні завади, механічний спектр або точність аналогових датчиків. Камера, SPI-flash і PWM не входять до цього стенда форми B. Тому їхні протоколи не подаються як перевірені.

Аргументація архітектурних рішень

MPU6050 має I2C для конфігурації та читання всіх осей одним burst. Швидкість 100 кГц достатня для 20-мс циклу без зайвого навантаження на шину. PIR дає окремий GPIO-фронт, LDR використовує ADC, тому їхні відмови й семантика не змішуються з

IMU. Світло або присутність людини самі по собі не доводять фізичне пошкодження.

Core налаштовує clock, AF і IRQ. Platform board містить апаратні обгортки; Platform runtime планує опитування та обмежує парсер UART. Devices працює через SensorBus callbacks. App обробляє нормалізовані вибірки, час і команди, не включає HAL. Напрямки залежностей показано на рис. 1; пунктир означає test doubles.

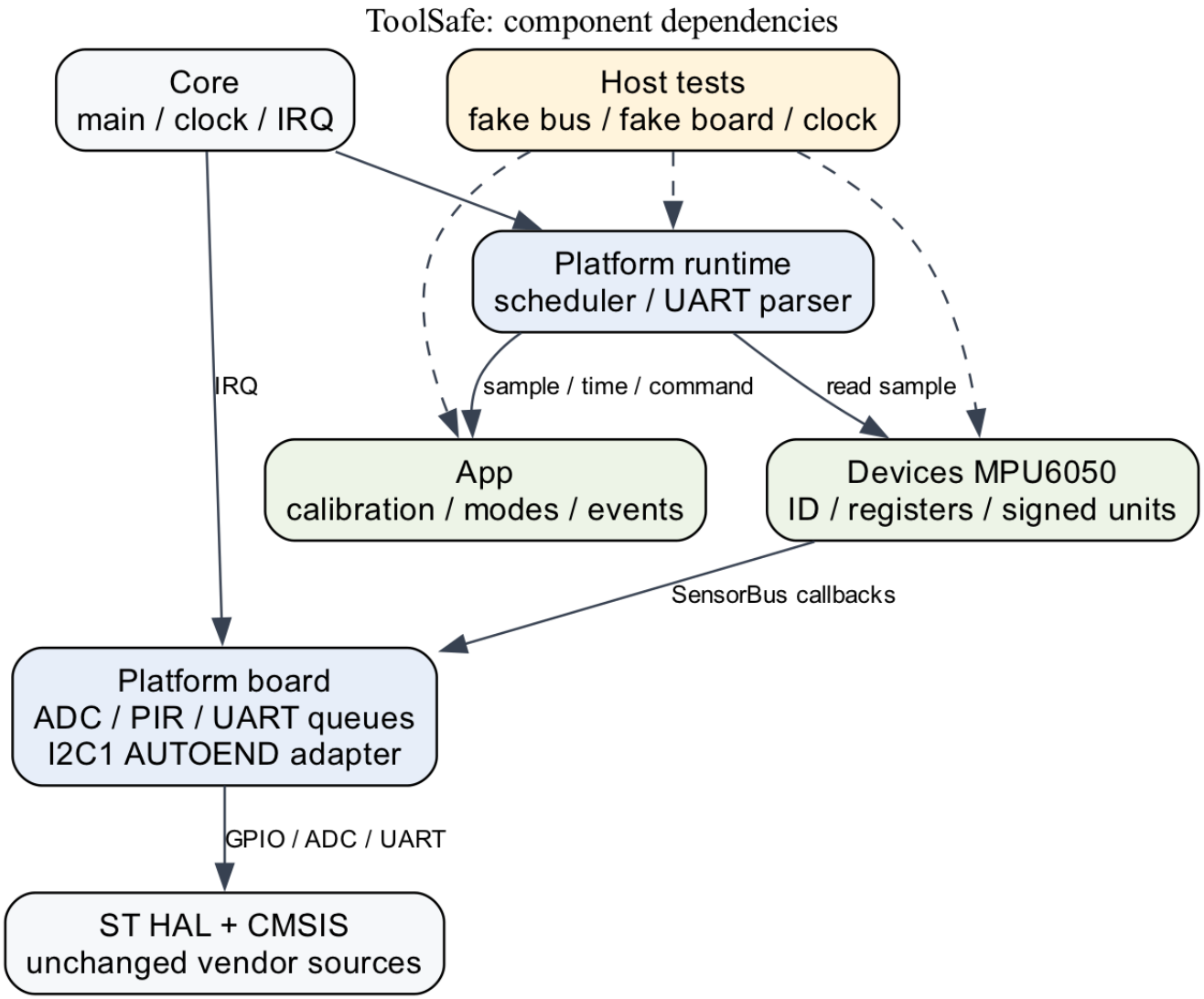


Рисунок 1 – Компоненти ToolSafe та напрямки залежностей

Pinout та інструкція підключення

Розкладку звірено з DS13867 Rev4: табл.12 с.31–34, табл.13 с.35, табл.15 с.36; фізичні роз'єми з UM2953 Rev1, рис.7 с.19, табл.11 с.20. Логіка та зовнішні підтяжки працюють від 3,3 В [1, 2].

MCU	Функція	Роз’єм Nucleo	Пристрій / аргумент
PB8	AF6 I2C1 SCL	D15, CN5.10; CN10.3	MPU SCL; штатний I2C, D0 аналізатора
PB9	AF6 I2C1 SDA	D14, CN5.9; CN10.5	MPU SDA; та сама шина, D1
PA0	ADC1 IN0	A0, CN8.1; CN7.28	LDR AO; окремий analog channel
PA1	GPIO input	A1, CN8.2; CN7.30	PIR OUT, pulldown; D3
PA2	AF1 USART2 TX	CN7.37, VCP	UART TX, D2; не Arduino D1
PA3	AF1 USART2 RX	CN10.10, VCP	UART RX, D6; не Arduino D0
PA4	GPIO output	A2, CN8.3; CN7.32	ADC marker, D4, push-pull
PA5	GPIO output	D13, CN5.6; CN10.11	Бортовий LED, D5

Послідовність підключення: вимкнути стенд; з’єднати всі GND; подати 3V3 на моделі MPU, LDR та PIR; AD0 MPU з’єднати з GND; PB8/PB9 під’єднати до SCL/SDA та додати дві підтяжки 4,7 кОм до 3V3; AO LDR до PA0, OUT PIR до PA1. Аналізатор під’єднати паралельно за таблицею, спільний GND обов’язковий. INT MPU не використовується. Після запуску очікується ID 0x68, валідні IMU/ADC та нульовий PIR у спокої.

У табл.12 с.23 UM2953 Rev1 позначення SCL/SDA біля PB8/PB9 переставлено відносно табл.11. У проєкті застосовано pin AF з datasheet та узгоджену табл.11. UART VCP PA2/PA3 також не тотожний Arduino D0/D1: останні належать USART1 PB7/PB6. PA13/PA14 лишено для SWD, PA5 зайнято LED; DMA і SPI1 не використовуються.

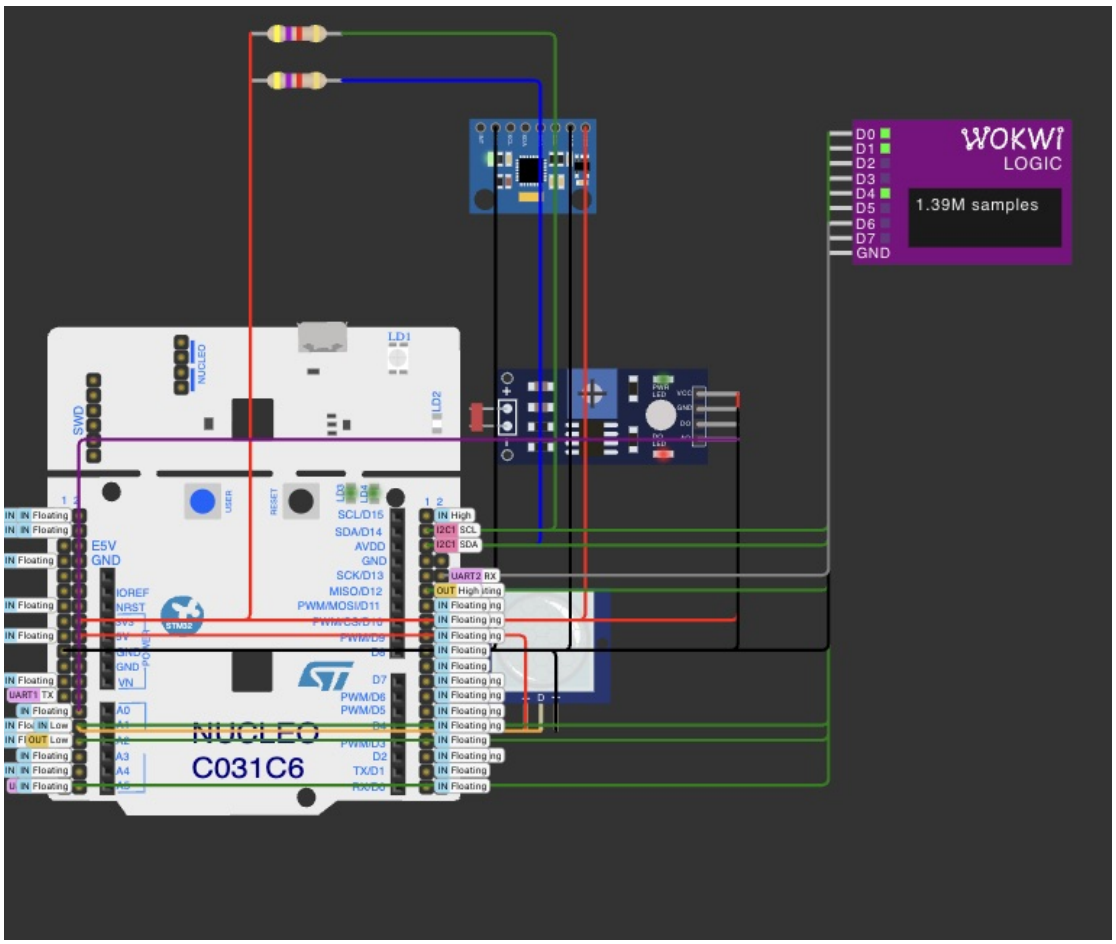


Рисунок 2 – Стенд ToolSafe: MPU6050, LDR, PIR та logic analyzer

Живлення, електричні обмеження та відхилені варіанти

MPU6050 потребує VDD 2,375–3,46 В; цифрові рівні прив’язані до VLOGIC. На SDA/SCL не подається 5 В [3]. AO фізичного LDR-модуля має бути в межах 0–3,3 В. Живлення PIR від 3V3 є параметром моделі Wokwi: для фізичного перенесення необхідно обрати документований модуль або перевірити його живлення й рівень OUT окремо.

Проєктний резерв для периферії: MPU 5 мА, LDR 5 мА, PIR 5 мА, LED 5 мА, дві підтяжки до 1,4 мА разом. Сума 21,4 мА округлена до 30 мА. Це резерви для вибору джерела, а не виміряні пікові струми конкретних модулів. UM2953 §6.5.2 обмежує суму плата+shield до 500 мА на 3V3. За планового резерву 100 мА для плати лишається 370 мА; цей розрахунок потребує перевірки амперметром, USB enumeration і температури регулятора. Силових виконавців немає.

Для підтяжки 4,7 кОм струм LOW приблизно 3,3/4700=0,70 мА на лінію. За припущення C=100 пФ оцінка $t_r = 0{,}8473RC \approx 0{,}40$ мкс менша за ціль 1 мкс standard-mode. Паралельні підтяжки з модулів зменшують еквівалентний опір і збільшують струм LOW. Wokwi дає ідеальні фронти; реальну ємність слід виміряти осцилографом.

Відхилено три варіанти: LDR і PIR одночасно на PA1 (analog/GPIO конфлікт); I2C SCL на PA5 (немає потрібної AF, pin зайнятий LED); VCP USART2 на Arduino D0/D1 (фізично інша USART). Окремі сигнали не ділять pin, периферію з несумісними режимами чи DMA-канал.

Частоти, ініціалізація та самоперевірка

SYSCLK, HCLK та PCLK дорівнюють 48 МГц. Регістри clock, I2C та ADC описано в RM0490 [4]. Для I2C1 залишено TIMINGR=0x20303E5D: PRESC=2, SCLH=62, SCLL=93. Сума програмованих high/low інтервалів $(63 + 94) \cdot 3/48$ МГц $= 9{,}8125$ мкс; до неї додаються фільтри та фронти. Власний VCD дає період 10 мкс, тобто 100 кГц. На фізичному MCU частоту слід переміряти, бо модель не відтворює всі аналогові затримки.

MPU адреса 0x68 за AD0=0, WHO_AM_I (0x75) повертає 0x68. Після ID послідовно записуються 0x6B=01, 0x19=09, 0x1A=03, 0x1B=00, 0x1C=00. Це PLL, DLPF=3, внутрішні 100 Гц, діапазони ±2 g та ±250°/c [5]. Прикладний цикл IMU 20 мс, ADC 20 мс; повільніший читач може пропускати внутрішні вибірки MPU, це не FIFO-запис усіх даних.

Команда `selftest` повторює ID/ініціалізацію та читання MPU. Стартовий рядок і періодичний status машинно читані: час, mode, imu validity, усі осі в mg/mdps, ADC raw/valid, PIR, фони, calibration, alarm, TX overflow. PIR та LDR не мають цифрового ID: їх перевірено за рівнем OUT, зміною AO та цифровим маркером вимірювання. Помилка одного каналу не зупиняє інші.

Запис та декодування I2C

У Wokwi використано logic analyzer, VCD timescale 1 нс [6]. Для коротких вікон I2C, UART та ADC імпорт sigrok виконується з 20 МГц (крок 50 нс); для довгих GPIO/ADC

вікон 1 МГц [7]. Співвідношення 200 вибірок на період I2C 100 кГц значно перевищує мінімум для логічного декодування. Апаратний запис на такій частоті за 1 с і 8 каналів потребував би близько 20 МБ сирих даних; VCD зберігає переходи, SR додатково стискається.

Захоплення стартує разом із симуляцією; це не умовний апаратний trigger. Потрібні вікна обрізані після запису. Початкові рівні ліній збережено, абсолютні offsets та SHA256 записані в `origin.json`. Природний фізичний trigger для I2C є START, для UART спад TX, для ADC фронт PA4. Повне джерело закінчується на 52.716224 с; пізніша робота термінала не доводить наявності відповідних фронтів у переповненому аналізаторі.

Декод WHO_AM_I: START, 0xD0, slave ACK, 0x75, ACK, STOP; START, 0xD1, ACK, 0x68, master NACK, STOP. PulseView показує 7-бітну адресу 0x68, тоді як 0xD0/0xD1 є адресними байтами з R/W. STOP між pointer write та read зберігає register pointer MPU. Повторний START тут не застосований.

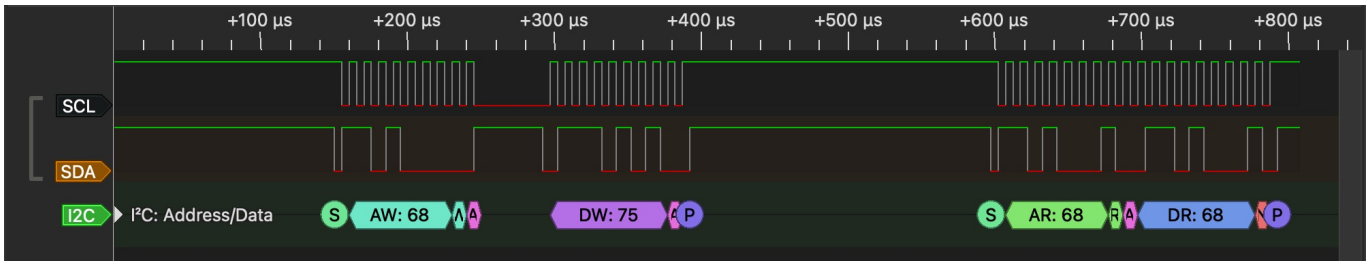


Рисунок 3 – Власне читання WHO_AM_I у PulseView; AW/AR=0x68, дані=0x68

Читання даних починається pointer 0x3B, далі 14 байтів за порядком accel X/Y/Z, temperature, gyro X/Y/Z. Власна послідовність у спокої:

```
00 00 00 00 40 00 EF 5C 00 00 00 00 00 00
```

Z=0x4000=16384 відповідає 1000 mg, модуль вектора у спокої 1,000 g (відхилення від заданої моделі 0%). Температура 0xEF5C=-4260 після цілочислового перетворення дає 24001 m°C для входу моделі 24°C. Це не оцінка точності реального термометра. Повна pointer+burst операція триває 1830.475 мкс, медіанний період burst 19.999350 мс.

Ручне декодування адреси

На сирій осцилограмі START є спадом SDA при високому SCL. Рівні SDA на перших восьми фронтах SCL: `1 1 0 1 0 0 0 0`, тобто 0xD0. Перші сім біт `1101000`=0x68, восьмий R/W=0. Дев'ятий рівень 0 є ACK від MPU. Байту register pointer 0x75 відповідає `01110101`; наступний ACK також 0. Ці висновки отримуються з рівнів ліній незалежно від назв анотацій. Ручний розбір відтворюється файлом i2c-manual-address.sr без протокольних анотацій; часовий offset є в однойменному origin.json.

Створений адресний NACK та відновлення

Команда `imuaddr 0x69` змінює програмну адресу, тоді як AD0 моделі лишається 0. На шині з'являється 0xD2, дев'ятий біт SDA=1: пристрій не підтвердив адресу. Операція триває 105.000 мкс. Це не master NACK останнього байта нормального читання. У status IMU стає invalid, фони анулюються; PIR, ADC та UART продовжують працювати. Спроби відновлення відбуваються раз на 1 с. Після `imuaddr 0x68` повертається WHO_AM_I та виконується калібрування за правилами профілю.

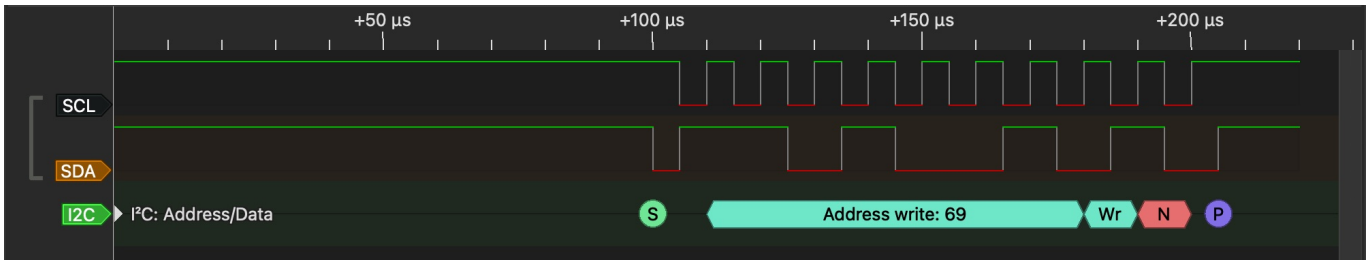


Рисунок 4 – Непідтверджена адреса 0x69: slave NACK

UART, ADC та GPIO

USART2 працює на 115200, 8N1. Один біт теоретично 8,6806 мкс, кадр 10 біт близько 86,806 мкс; між кадрами можливі паузи IRQ-черги. На рис. нижче PulseView у форматі ASCII показує початок рядка `seq=1`. Повні рядки незалежно від GUI декодовані з TX у `wire-uart.txt`; вони збігаються з журналом Wokwi.

RX і TX реалізовані окремими IRQ-чергами 256 і 2048 байтів. Переповнення відстежується, рядок понад 80 символів або з NUL відкидається до нового рядка. Парсер має

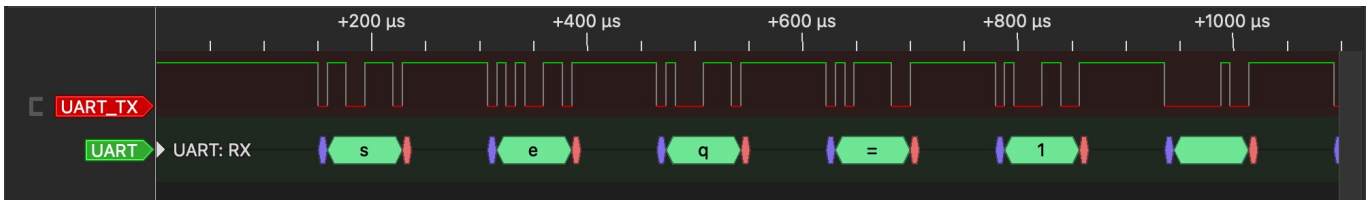


Рисунок 5 – ASCII-декод початку seq=1, UART 115200 8N1

бюджет 128 символів за ітерацію; sensor polling не затримується довільно довго через потік команд. У виконаних сценаріях TX overflow events=0.

ADC clock=PCLK/2=24 МГц, 12 біт, sampling 160,5 cycles. Теоретичні вибірки та перетворення $(160{,}5+12{,}5)/24$ МГц дають 7,21 мкс. PA4 огортає HAL Start/Poll/Get/Stop, тому виміряна ширина 65.225 мкс включає програмні накладні витрати та модель. Це не апертурний час ADC. Медіанний період PA4 19.999575 мс відповідає налаштованим 20 мс.

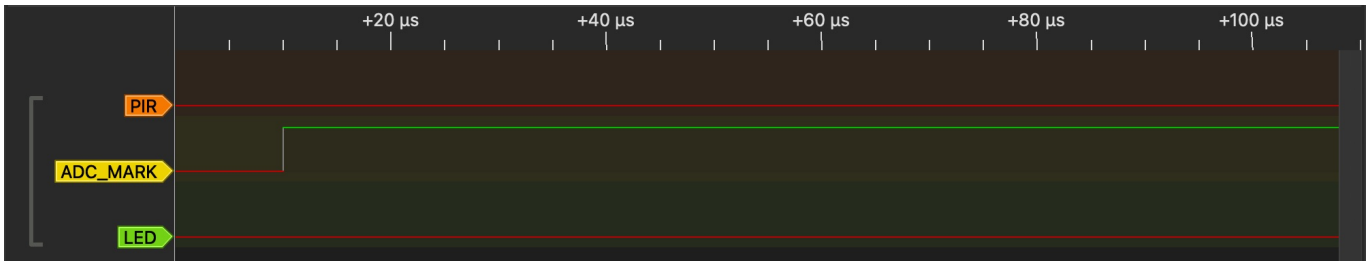


Рисунок 6 – PA4 огортає програмне вимірювання ADC; крок SR 50 нс

У нормальному освітленні 10 lux raw=3413; після 100000 lux raw=32. Значення в межах 0–4095 та правильний напрям зміни підтверджують логіку моделі LDR. Калібровані lux, нелінійність і реальну точність ADC не визначено. Вхідний фільтр використовує п’ять вимірів, тому event value може відрізнятись від останнього raw.

PIR HIGH на VCD триває 5.000000025 с. Рівень HIGH створює одну подію на фронт, не нову подію в кожному циклі. LED у нормі перемикається раз на секунду; за відмови блимає швидше, за тривоги світиться стало. Це цифрова індикація, не силовий виконавець.

Прикладна логіка та одночасні впливи

Режими: `guard, issue`. Після запуску та відновлення датчика автоматично збираються 100 вибірок, близько 2 с. Команди guard та issue підтримуються без префікса mode. Зміна модуля прискорення понад 0,8 g дає impact; повернення нижче 0,5 g переозброює детектор. Векторна швидкість понад 30°/с дає body_motion, повернення нижче 20°/с знімає активний стан. Світло усереднюється за п’ятьма вибірками: зміна 15% ADC, повернення до 10%. PIR означає підхід, не відкриває сейф і не доводить авторизацію. У guard механічні та світлові події тривожні, в issue інформаційні.

Для одночасного тесту симуляцію призупинено, в MPU встановлено X=2 g при Z=1 g, у LDR 100000 lux та натиснуто PIR Simulate motion; далі виконання відновлено. Ступінчастий вектор є керованим тестом моделі, а не каліброваним фізичним ударом чи чистим статичним нахилом. X насичується на 32767 LSB, тобто 1999 mg. На 45,832 с PIR дав approach (INFO); через 3 мс з’явилася light_change (ALARM), через 16 мс impact (ALARM). Значення impact=1235 mg є різницею модулів, а не самою компонентою X=1999 mg. Світловий детектор спрацював на середньому 2736, тоді як остання сира вибірка стала 32. Переходи guard→issue→guard виконані в нормальному стані.

Час MCU, мс	Джерело	Подія	Рівень / value
45832	GPIO	approach	INFO / 1
45835	ADC	light_change	ALARM / 2736
45848	IMU	impact	ALARM / 1235

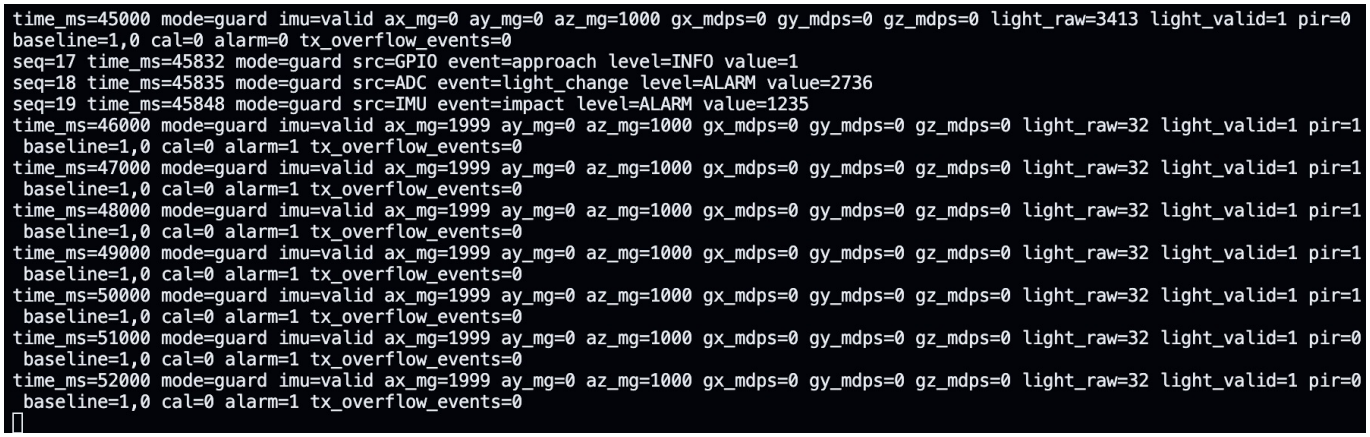


Рисунок 7 – Реальний журнал одночасних подій ToolSafe

Повні event rows мають монотонний seq і time_ms, тому кілька каналів не маскують один

одного. Час MCU округлений до мілісекунд; час VCD має інший початок. Порівнювати їх як дві ідентичні абсолютні шкали не можна. Для оцінки затримки враховуються квантування MCU та черга UART, а аналоговий початок LDR без окремого аналогового вимірювання не спостерігається.

Структура, бібліотеки та автоматичні перевірки

Вендорські CMSIS і STM32C0 HAL з базової ЛР1 збережено. Додані модулі ADC, ADCEx і LL_ADC версії HAL v1.0.1 контролюються SHA256-manifest. HAL модулі: RCC/FLASH для clock, GPIO/EXTI для pin і вендорських залежностей, CORTEX/PWR для core, USART для UART, I2C для базової конфігурації, ADC для аналогового каналу; DMA модуль лишився частиною конфігурації базового HAL, але DMA transfers не використовуються. Перевірка `tools/check_project.py` забороняє HAL/MCU includes у переносних App/Devices і перевіряє незмінність доданих vendor files.

Адаптер I2C1 у Platform використовує AUTOEND і цілі записи CR2, deadline 10 мс та settle interval 120 мкс після старту/передачі/STOP. Це виправлення спостереженої поведінки моделі; на фізичному MCU треба повторити вимірювання. HAL sources не змінювалися. За помилки очищуються flags та CR2, за потреби перемикається PE. Після відмови ADC застосовується повторна ініціалізація через 1 с і п’ять нових вибірок фільтра.

Перетворення MPU у фізичні одиниці є переносним C-кодом:

```
static int32_t signed_word(const uint8_t *data) {
    uint32_t word = ((uint32_t)data[0] << 8) | data[1];
    return word < 32768u ? (int32_t)word : (int32_t)word - 65536;
}

void mpu6050_decode(const uint8_t bytes[14], ImuSample *sample) {
    for (size_t axis = 0; axis < 3; ++axis) {
        sample->accel_mg[axis] = signed_word(bytes + 2 * axis) * 1000 /
   16384;
        sample->gyro_mdps[axis] = signed_word(bytes + 8 + 2 * axis) * 1000 /
   131;
    }
    sample->temperature_mc = signed_word(bytes + 6) * 1000 / 340 + 36530;
}
```

`make build` виконується в Docker із pinned base image, ARM GCC та -Os -Wall -Wextra.

Конкретні версії закріплено у Dockerfile, без залежності від поточного системного компілятора. GitHub Actions запускає build, Linux tests і coverage на branch/PR; checkout action pinned commit. Вихідний ELF/HEX та logs збережено, HEX після повторної збірки збігся із записаним у capture manifest.

Результати тестування

Використано п’ять assert-based suites: MPU driver, calibration, events, acceptance boundaries та runtime. Fake I2C перевіряє ID, порядок усіх п’яти config writes, NACK, timeout, unexpected ID і незмінність output sample за помилки. Тести калібрування охоплюють точну кількість вибірок, шум, векторну швидкість, переривання та окремі фони. Події перевіряють hysteresis, межі порогів, одночасність каналів, quiet interval і wraparound часу. Runtime працює з fake Board/clock/UART: перевірено retry не частіше 1 с, незалежність PIR/ADC, CRLF, overflow та selftest failure.

Два вектори `tests/fixtures/mpu-captured.h` походять із власного VCD: у спокої Z=1000 mg; після впливу X=1999 mg. Для кожного fixture збережено source hash і час транзакції. Ці самі компоненти показує UART. Температура перевірена за формулою та входом моделі; окремого фізичного термометра не було. Assert достатній для незалежного host runner, проте не надає автоматичної генерації mocks і багатого framework report.

Пороги coverage обрано 85% рядків і 70% гілок: переносна логіка має покриватися широко, але число не підміняє перевірку апаратних шляхів. Виміряні результати наведено нижче. Board/HAL не включено до цих відсотків; runtime перевірений fake-тестами окремо, а апаратний шлях моделлю Wokwi.

Модуль	Рядки виконано / усі	Гілки taken / усі
app.c	74 / 74	88 / 90
commands.c	77 / 89	94 / 162
events.c	76 / 77	135 / 169
mpu6050.c	23 / 23	14 / 14
Разом	250/263 (95.06%)	331/435 (76.09%)

Остаточні локальні build, Linux tests і coverage завершилися з кодом 0. ASan/UBSan пройдено на етапі граничних тестів. CI та посилання на гілку: https://github.com/FA1RY2412/ToolSafe/tree/feature/lab-2. Захвати відповідають перевіреним source та

HEX; dates у metadata Git не використовуються як годинник вимірювань.

Журнал проблем та виправлень

Наведені проблеми стосуються коду, інструментів перевірки та документації. Свідомо створений NACK позначений окремо від дефектів.

1. **Неузгодженість адаптера та pin AF.** Попередній board layer керував I2C як GPIO, тоді як базова конфігурація pin визначала I2C1 AF6. Виправлено апаратним AUTOEND-адаптером і цілими записами CR2; власні фінальні SCL/SDA підтверджують транзакції. Коміт 342efc1.
2. **Гонка передавання моделі.** ID міг читатися, але наступна config write не гарантувала правильний STOP. Після START, TXDR та STOP додано обмежений settle interval, очищення flags/CR2 і deadline. Повторне ID та burst працюють. Коміт 342efc1; це обґрунтування адаптації моделі, не вимірювання реального MCU.
3. **Створений address NACK.** Адреса 0x69 не відповідала AD0=0. Дев'ятий біт HIGH підтверджує відмову; повернення 0x68 відновлює sensor і запускає потрібне калібрування. Відмова не зупиняє інші канали. Обробка cdbffb3, signal-level перевірка 342efc1.
4. **Runtime test втрачав printf output при оптимізації.** На Linux FORTIFY міг перенаправляти stdio, через що macro-підміна printf не бачила calibration_ready. Додано явний test-only seam RUNTIME_TEST_PRINTF після включення stdio; production UART не підміняється. Коміт b703911.
5. **Coverage runner не компілювався з -Werror.** Після перейменування test main у unit function неявне повернення 0 більше не мало спеціального значення C main. Додано явний return0; усі suites об'єднуються без придушення warning. Коміт b703911.
6. **Суперечливі підписи в UM2953 Rev1.** У табл.12 SCL/SDA переставлено. Звірено DS13867, табл.11 та рис.7; у pinout зафіксовано PB8=SCL, PB9=SDA. Коміт 342efc1.

Відповідальність за напрями

Напрям	Відповідальний	Артефакти
Звіт і документація	Голуб Д.Д.	PDF, джерело звіту
Pinout і джерела	Голуб Д.Д.	pinout, diagram.json
Стенд та запис	Голуб Д.Д.	Wokwi, VCD/SR
Декодування й аналіз	Голуб Д.Д.	PVS, text decodes
Структура і збірка	Голуб Д.Д.	Core/Platform/Devices/App, CI
Тестування	Голуб Д.Д.	fake suites, captured vectors, coverage

Висновки

У ToolSafe одночасно працюють MPU6050, PIR та LDR. WHO_AM_I=0x68, спокійний вектор дорівнює 1 g; сигнали та декод UART підтверджують узгодженість моделі й прикладних станів. I2C дає 100 кГц, IMU опитується приблизно кожні 20 мс, ADC кожні 20 мс. Власний address NACK призводить до invalid state і контрольованого відновлення, а не до зупинки всіх каналів.

Прикладні правила залежать від режиму та зберігають незалежність сенсорів. На 45,832 с PIR дав approach (INFO); через 3 мс з'явилася light_change (ALARM), через 16 мс impact (ALARM). Значення impact=1235 mg є різницею модулів, а не самою компонентою X=1999 mg. Світловий детектор спрацював на середньому 2736, тоді як остання сира вибірка стала 32. Переходи guard→issue→guard виконані в нормальному стані. Переносні App і Devices покриті на 95.06% рядків та 76.09% гілок. Повторна збірка дала той самий HEX, що й під час запису.

Перевірка доводить цифрову інтеграцію у Wokwi. Вона не доводить фізичну надійність охоронного пристрою, метрологічну точність, реальні піки струму чи коректність фронтів на breadboard. Для перенесення на залізо потрібні контроль живлення, струму, pull-ups і VCP bridges, аналогове вимірювання та повторний capture тим самим набором тестових дій. Відомий settle interval I2C є адаптацією моделі та підлягає повторній оцінці.

Список використаних джерел

[1] STMicroelectronics. *DS13867 Rev4. STM32C031x4/x6 datasheet*. URL: https://www.st.com/resource/en/datasheet/stm32c031g6.pdf.

[2] STMicroelectronics. *UM2953 Rev1. STM32 Nucleo-64 boards MB1717*. URL: https://www.st.com/resource/en/user_manual/um2953-stm32-nucleo64-board-mb1717-stmicroelectronics.pdf.

[3] InvenSense. *MPU-6000/MPU-6050 Product Specification, Rev3.4*. 2013. URL: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Datasheet1.pdf.

[4] STMicroelectronics. *RM0490 Rev5. STM32C0x1 reference manual*.

[5] InvenSense. *MPU-6000/MPU-6050 Register Map and Descriptions, Rev4.2*. 2013. URL: https://invensense.tdk.com/wp-content/uploads/2015/02/MPU-6000-Register-Map1.pdf.

[6] Wokwi. *Logic Analyzer and VS Code Project Configuration*. URL: https://docs.wokwi.com/guides/logic-analyzer (дата зверн. 15.10.2026).

[7] sigrok project. *PulseView and protocol decoders*. URL: https://sigrok.org/wiki/PulseView (дата зверн. 15.10.2026).